

Self Reflection

Jenith Gurung | Student ID: 25068608

My Role in the Project

I was responsible for the backend foundation of the Student Profile Management System. This meant building the three core files that the rest of the team depended on: `main.py`, `auth.py`, and `file_handler.py`. My work had to be completed first since every other team member's module relied on the file handling and login system I built.

What I Built

`file_handler.py`

This module handles all reading and writing across the four data files: `users.txt`, `passwords.txt`, `grades.txt`, and `eca.txt`. I wrote consistent read and write functions for each file, along with lookup functions such as `get_user_by_username` and `get_grades`, which teammates could import and use directly without needing to handle file parsing themselves.

`auth.py`

This module handles the login system. I designed a `User` class as the base, with `Student` and `Admin` subclasses that inherit from it — this satisfied the OOP and inheritance requirements from the brief. The login function validates credentials against `passwords.txt`, looks up the matching user record, and returns the correct subclass depending on the role. It also enforces a three-attempt limit before exiting.

`main.py`

This is the entry point of the program. After a successful login it routes the user to either the admin panel or the student menu depending on their role. I used `try/except ImportError` blocks so the program would not crash if a teammate's file was not yet ready, which made integration easier during development.

Challenges I Faced

The main challenge was the data format. Our original team plan had `users.txt` with three columns, but once I started building the login system I realised there was no way to map a username like `ZENITH_G` to a student ID like `101` without a fourth column. I added a username field to `users.txt` and updated `file_handler.py` accordingly, then had to communicate this change to the team so their code would not break.

Integration was also harder than expected. When I reviewed my teammates' files before submission, I found that `student.py` was importing functions that did not exist in `file_handler.py`, and `admin.py` was running its menu loop at import time rather than inside a callable function. I worked through the fixes with the team over the group chat on the day of submission.

What I Learned

- Designing shared modules is harder than writing code just for yourself — every function name, return type, and file format becomes a contract with your teammates.
- Exception handling is worth doing even for simple file operations. The `try/except` blocks I added saved a lot of confusion during testing when files were missing or had the wrong format.
- OOP makes code easier to extend. Having `User` as a base class and `Student/Admin` as subclasses meant the role logic stayed clean and easy to follow.
- Communication in a group project matters as much as the code itself. The data format change I made to `users.txt` needed to be clearly explained so nobody's module broke.

What I Would Do Differently

I would agree on the exact file formats and function names with the team before anyone wrote a single line of code. A lot of the integration problems we ran into on the last day came down to assumptions — Aayetish assumed `file_handler` had a `read_file` function, Abijeet's admin panel ran at module level rather than inside a function. A short planning document at the start would have avoided most of that.

Overall

I am satisfied with what I built. The backend is clean, well-structured, and handles errors properly. It gave the rest of the team a solid foundation to build on and the system came together as a working whole by the end. Group projects are always a balance between your own code quality and making sure everything connects, and I think this project taught me a lot about both sides of that.